

Banco de Dados PostgreSQL

Como os dados de vendas, cupons e cadastro das máquinas foram estruturados no Postgres, e como essa estrutura resolve os erros de faturamento e ciclos que apareciam no painel.

12

TABELAS

3

GARANTIAS CENTRAIS

496

VENDAS MIGRADAS

100%

CONFERIDO COM DADOS REAIS

● Fase 2 concluída — leitura ativa no painel

O problema que motivou a mudança

O painel mostrava números de vendas e ciclos que não batiam com o que as máquinas realmente processavam. A investigação encontrou **três defeitos independentes se somando** — e os três nasciam da mesma causa: o sistema guardava tudo num arquivo de texto (JSON), sem nenhuma regra que impedisse dado errado de entrar.

Nº	DEFEITO	EFEITO NO PAINEL
1	Uma venda podia ser gravada duas vezes — uma confirmação vinda da máquina e outra vinda do provedor de pagamento, sem checagem de duplicidade.	Faturamento e número de ciclos apareciam em dobro.
2	O sistema calculava "hoje" usando o horário de Greenwich, não o de Fortaleza.	Vendas feitas depois das 21h eram contadas no dia seguinte.
3	A modalidade da higienização era gravada com grafias diferentes conforme de onde vinha a confirmação (com acento, sem acento, abreviada).	O gráfico por modalidade ignorava até 61% das vendas do dia.

Evidência real, colhida em produção

O painel exibia **4 ciclos / R\$ 60,00** como faturamento de "hoje". Na prática eram **2 vendas reais** (contadas em dobro), ocorridas às 21h25 e 21h26 *do dia anterior*. O valor correto de "hoje" naquele instante era zero.

Um arquivo de texto não tem como impedir isso — qualquer parte do código pode escrever ali sem passar por nenhuma verificação. Um banco de dados relacional como o PostgreSQL resolve isso **por construção**: a regra fica no próprio banco, e nenhum caminho de código consegue burlá-la.

As três regras que tornam os defeitos impossíveis

Cada um dos três defeitos acima virou uma regra escrita diretamente na estrutura das tabelas. O banco passa a **recusar** o dado errado, em vez de aceitar e deixar o erro se espalhar pelo painel.

Regra 1

Venda duplicada é rejeitada

Um índice único garante que o mesmo pedido, na mesma máquina, só pode existir uma vez como venda aprovada.

```
UNIQUE (order_id, devno)
WHERE status = 'APPROVED'
```

Regra 2

Modalidade inválida é rejeitada

A coluna só aceita três valores possíveis — qualquer grafia diferente é recusada na entrada, não silenciosamente ignorada depois.

```
CHECK (mode IN (
  'BASICA', 'INTERMEDIARIA',
  'AVANCADA'))
```

Regra 3

O dia é sempre o de Fortaleza

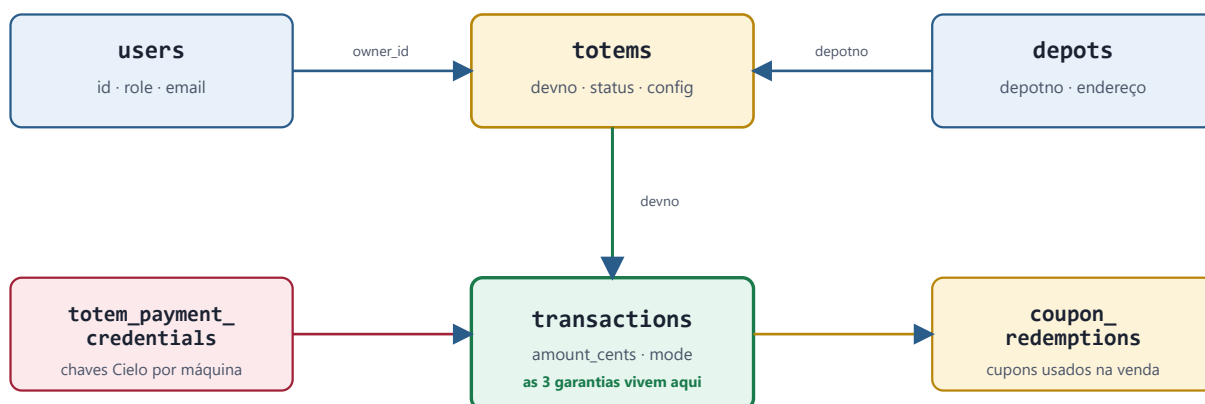
Uma visão pronta do banco já devolve a data no fuso correto — nenhum relatório precisa calcular isso na mão de novo.

```
(occurred_at AT TIME ZONE
  'America/Fortaleza')::date
AS business_date
```

Essas três regras foram testadas de propósito, tentando quebrá-las: duas vendas com o mesmo pedido → a segunda é recusada pelo banco. Uma modalidade fora da lista → recusada. Uma venda simulada às 21h30 de Fortaleza → o banco a coloca corretamente no dia local, não no dia seguinte em UTC.

As doze tabelas e como elas se conectam

O banco tem **12 tabelas**, organizadas em quatro grupos: cadastro (quem são os donos, as máquinas e os pontos de instalação), vendas, cupons e operação. O diagrama abaixo mostra o núcleo — como vendas se ligam a máquinas, donos e pontos.



Uma máquina (`totems`) pertence a um dono (`users`) e fica instalada num ponto (`depots`). Toda venda (`transactions`) referencia a máquina onde aconteceu — é aí que as três garantias da seção 2 se aplicam. Cada máquina tem suas próprias credenciais de pagamento, guardadas separadas da tabela principal.

Por que credenciais de pagamento viraram uma tabela à parte

Antes, o controle de "quem pode ver a chave da Cielo" era feito apagando campos de um objeto já carregado — um código específico decidia, requisição a requisição, quais campos remover antes de responder. Agora o controle vira uma regra do próprio banco: quem não é administrador simplesmente não tem permissão para consultar `totem_payment_credentials`. É mais simples e não depende de nenhum código lembrar de mascarar o dado toda vez.

As outras oito tabelas

branches

CADASTRO

Filiais da rede. Hoje existe uma só (matriz).

alerts

OPERAÇÃO

Avisos técnicos — nível de sanitizante baixo, ordens de manutenção abertas.

alert_comments

OPERAÇÃO

Histórico de comentários numa ordem de manutenção.

coupons

CUPONS

Cadastro dos cupons: desconto, limite de usos, máquinas permitidas.

pending_orders

OPERAÇÃO

Pagamentos em andamento (PIX gerado, cartão em processamento). Antes só existia na memória do servidor — se o servidor reiniciasse no meio de um pagamento, o registro sumia e o estorno automático não sabia que precisava agir.

system_settings

OPERAÇÃO

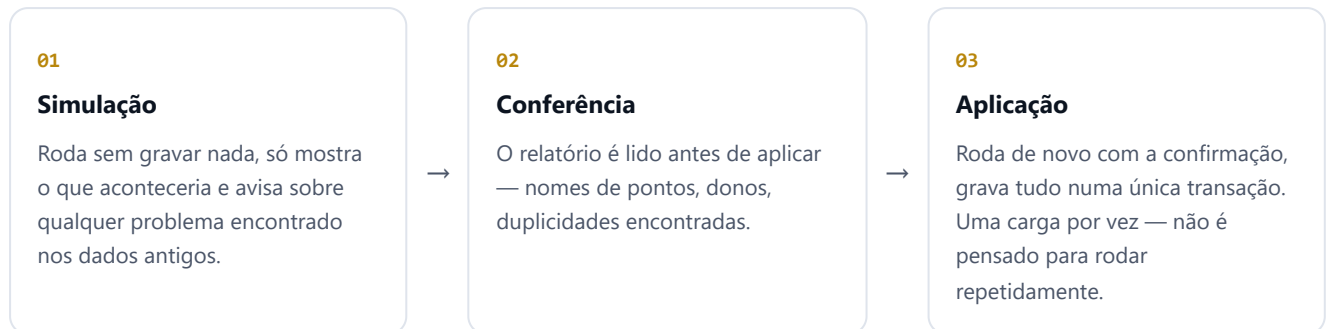
Configuração única de credenciais padrão da rede.

Dinheiro sempre em centavos, nunca em decimal

Todo valor monetário é guardado como número inteiro de centavos (`amount_cents`), não como "R\$ 15,00". Isso elimina de vez os erros de arredondamento que aparecem quando um sistema faz conta com casas decimais.

A migração dos dados existentes

A rede já tinha meses de vendas, cadastros e cupons guardados no arquivo antigo. Um script de importação (`scripts/import_json.js`) lê esse arquivo e grava tudo no Postgres, dentro de **uma única transação**: se qualquer coisa falhar no meio, nada fica gravado pela metade.



Rodando essa importação contra os dados reais da produção, o script encontrou e corrigiu sozinho quatro problemas que só apareceriam com dados de verdade:

PROBLEMA	O QUE ERA	COMO FOI RESOLVIDO
Ponto duplicado	Dois pontos de instalação diferentes (endereço real distintos) tinham acabado com o mesmo código, por causa de um defeito antigo no gerador de códigos.	Mantém o mais antigo no código original, dá um código novo para o mais recente — nenhum dos dois é perdido.
Alerta duplicado	Dois avisos técnicos reais, de dias diferentes, tinham o mesmo código de identificação.	Mesma lógica: nunca descarta, gera um código novo para o registro mais recente.
Nome do ponto genérico	Todos os 13 pontos de instalação da rede estavam salvos com o nome genérico "Ponto de Instalação" em vez do nome real (ex.: "Merit Offices & Mall").	O script recupera o nome real de um campo alternativo onde ele efetivamente estava.
Dono não identificado	Quando uma máquina aponta para um dono que não existe mais no cadastro.	Atribui ao administrador da rede e registra o motivo no relatório, em vez de travar a importação inteira.

Conferência financeira

A soma de todas as vendas aprovadas, depois de remover as duplicatas, bateu **exatamente** entre o arquivo original e o banco novo: 496 vendas, R\$ 3.305,29, nos dois lados.

O que já funciona com o banco novo

O site continua funcionando **normalmente sobre o sistema antigo** para registrar vendas do dia a dia — essa troca é uma etapa futura, feita aos poucos e sem risco (seção 6). O Postgres já tem uma função ativa hoje: a aba "**Histórico (arquivo)**", dentro da tela de detalhes de cada máquina.

Como funciona

Ao abrir uma máquina no painel e clicar em "Histórico", o site consulta o Postgres e mostra: todas as vendas já registradas naquela máquina, um resumo (total vendido, ticket médio, dias com movimento) e os cupons resgatados nela. É uma **fotografia** tirada no momento em que os dados foram carregados — não atualiza sozinha a cada venda nova.

O que acontece com uma venda feita depois da fotografia

Ela continua aparecendo normalmente na tela principal (que usa o sistema atual), e a aba de Histórico avisa quantas vendas "novas" ainda não entraram no arquivo — em vez de simplesmente não mostrar o número e deixar o usuário achar que uma venda sumiu.

Exemplo — arquivo em dia

"Arquivo congelado em 09/09, 14:20. Cobre vendas até 07/09, 13:21. **Arquivo em dia** — nenhuma venda posterior ao arquivo."

Exemplo — vendas fora do arquivo

"Arquivo: 94 vendas (R\$ 505,09). **Fora do arquivo: 3 vendas (R\$ 45,00)** — aparecem na aba Visão geral."

Se o banco não estiver disponível

O site inteiro **não depende** do Postgres para continuar funcionando. Se o banco não estiver configurado no servidor, a aba de Histórico mostra um aviso simples explicando isso, e o resto do painel — vendas do dia, ciclos, alertas — continua operando sem nenhum efeito colateral.

Roteiro até o Postgres virar a fonte principal

A troca completa — o site passar a gravar cada venda direto no Postgres, em vez do arquivo antigo — é feita em etapas pequenas e reversíveis, para nunca arriscar uma venda em andamento numa máquina real.

ETAPA	O QUE ACONTECE	RISCO
Feito	Banco criado, testado e importado com os dados reais. Site lê o Postgres só para o histórico por máquina.	Nenhum — nada em produção depende disso ainda.
Próxima	O site passa a <i>também</i> gravar cada venda nova no Postgres, mas continua usando o arquivo antigo como fonte oficial — só para comparar os dois e confirmar que batem, dia após dia.	Baixo — se a gravação no Postgres falhar, a venda no sistema antigo continua valendo normalmente.
Depois	Com dias seguidos sem nenhuma diferença entre os dois, o Postgres vira a fonte oficial de leitura. O arquivo antigo continua sendo escrito, como segurança.	Baixo — reverter é uma variável de configuração e reiniciar o processo.
Por fim	O arquivo antigo para de ser escrito. O Postgres passa a ser o único lugar onde os dados existem.	Médio — só acontece depois de um export de segurança testado e confirmado.

Capaxero Cloud — documentação técnica gerada a partir do estado real do banco de dados em 10/09/2026. Detalhes de implementação, comandos e histórico completo de decisões ficam em `MIGRACAO.md`, no repositório do projeto.